

Team Member Full Name	NetID
Danny Gallagher	dgallag5
Sam Gisinier	sgisinier

Persistent Storage Design

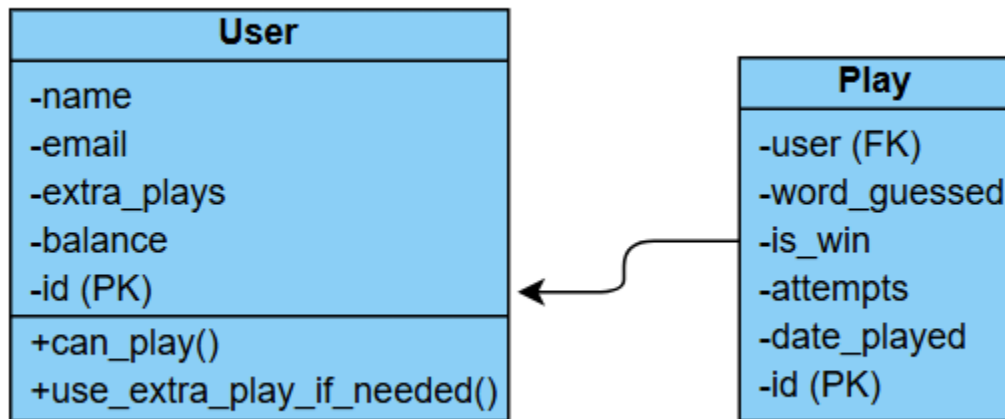


Figure 1 database schema

The above UML diagram depicts and summarizes the way in which we store data for our web application. We use SQLite, and we have implemented two models, User and Play. User extends Django's AbstractUser class and includes more attributes and methods that are specific to our use of the class in this application, like the number of extra plays the user has purchased and possesses. The Play model contains information about a specific game of WorNDly that a user has played, such as when the game was played, how many attempts the user took to guess the word, and so on. These two models have a one-to-many relationship; a user can have many plays because a person can play the game multiple times, while each play belongs to exactly one user. This relationship is implemented using a ForeignKey in the Play model that references the user that the play instance belongs to. The Play model uses cascade deletion as well, because if a User is deleted, all their plays should also be deleted.

Demonstration of the Features

Feature 1.1

Feature 1.1 necessitates the website allowing people to make a player profile that has a name, username, email, and password. Figure 2 shows the screen that is used to sign up, enabling the

user to create a new account. There is also a link on the screen to go to the login page. When the user inputs their information into all the fields, it will be saved to the User model, which is a subclass of the AbstractUser class. Then, after the user creates their profile, they will be redirected to the login screen.

Sign Up

Name (Optional):

Username:

Email:

Password:

[Log in](#)

Figure 2 Screenshot for feature 1.2 showing the sign up screen

Feature 1.2

Feature 1.2 requires that users are able to login to the website using their username and password. Figure 3 shows the screen that is used to login, enabling the user sign in to their account. There is also a link on the page allowing the user to go to the sign up page in case they have not created an account yet. The user is able to put their username and password in the form and then submit it. Once the form is submitted, the user will be authenticated (correct username and password) and will be directed to the home page if they log in successfully. If not, they will just be redirected to the login page again. Additionally, a user can only access the home page if they are logged in.

Login

Username:

Password:

Login

[Sign up](#)

Figure 3 screenshot for feature 1.3 showing the login page

Feature 1.3

Feature 1.3 requires users to be able to logout of their session. Figure 4 shows the current home page of the site that the user is directed to once they are logged in, as well as the logout option they can select. When a user clicks this link, their login session will end and they will be redirected back to the login page. They will no longer be able to access the home page until they log in again, because that page is login-required.

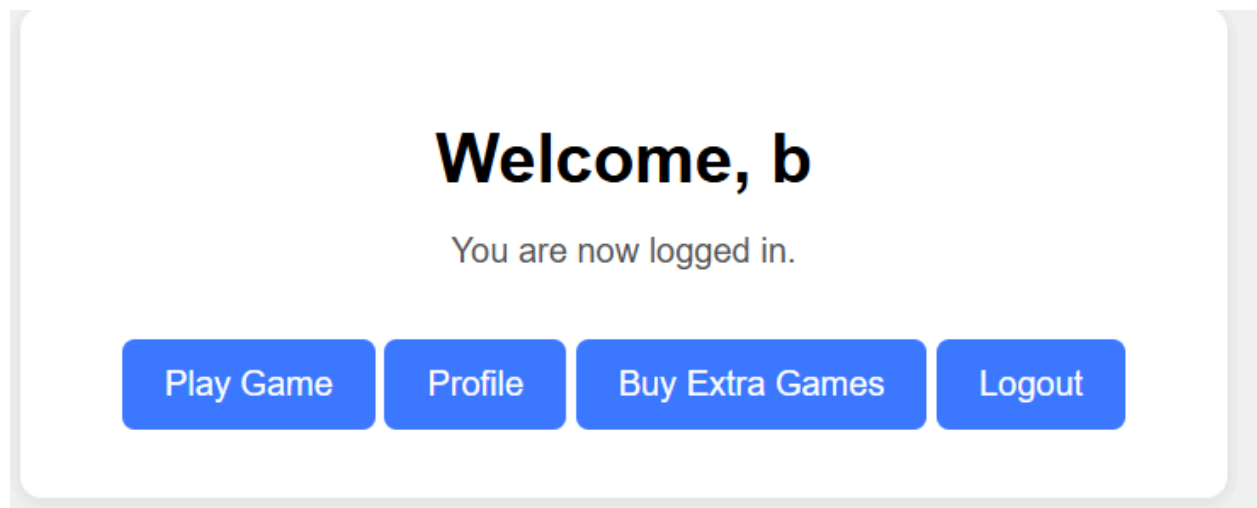


Figure 4 screenshot for feature 1.3 showing the home page that includes a logout link

Feature 2.1

Feature 2.1 is the main part of this project where the game is actually played. The user first is able to select the language of the game they want to play. Once they select the language, it checks whether the user has enough plays left, either for the day or with their purchased plays. Figure 5 shows the site before any language has been selected. Once the language is selected and the user is deemed able to play, the game will appear. Figure 6 shows how the game looks as it is being played. The user can enter a 5 letter word in the text box and it will populate on the game board once it is validated to be a valid 5 letter word of the language. The correct colors for each letter will be the background color of the letter as they appear on the board. If the user guesses correctly, they are informed with the number of guesses it took, and if they cannot guess the word they are told what the word is. Once the game finishes, they are able to select a language again to play a game.

Play Game

Welcome to the game!

DE German

ES Spanish

FR French

BR Portuguese

us English

[Home](#)

Figure 5 screenshot for feature 2.1 showing the game before it is played

Play Game

Welcome to the game!

DE German

ES Spanish

FR French

BR Portuguese

us English

S	L	A	T	E
C	A	P	E	R

Submit Guess

[Home](#)

Figure 6 screenshot for feature 2.1 showing the game in action

Feature 3.1

Feature 3.1 allows the user to look at all of their previous plays and also filter by different time periods. When they go to their profile, it will automatically show them all of their previous games played with the word, the date, the outcome, and the number of attempts it took. The user can also click to filter by games played in the last week, month, year, or all time. Figure 7 shows how the site will look to the user when they initially go to the profile page.

Profile

Plays

Last Week Last Month Last Year All Time

- BOSON - Win (in 1 attempts) on 2026-04-28 22:15
- FIRIE - Loss on 2026-04-28 21:48
- ABAND - Win (in 1 attempts) on 2026-04-28 21:42
- THUGS - Win (in 2 attempts) on 2026-04-28 21:40
- BUFFI - Win (in 1 attempts) on 2026-04-28 21:25

Figure 7 screenshot for feature 3.1 showing the stats of all previous games played

Feature 3.2

Feature 3.2 allows the user to see some statistics about their plays of the game. They are able to see the total number of times they have played, their win rate, and a distribution of the amount of times each win took a certain number of attempts. Figure 8 shows a sample of what this part of the profile looks like to a player who has played 5 games before.

Stats

Total Plays: 5

Win %: 80.0%

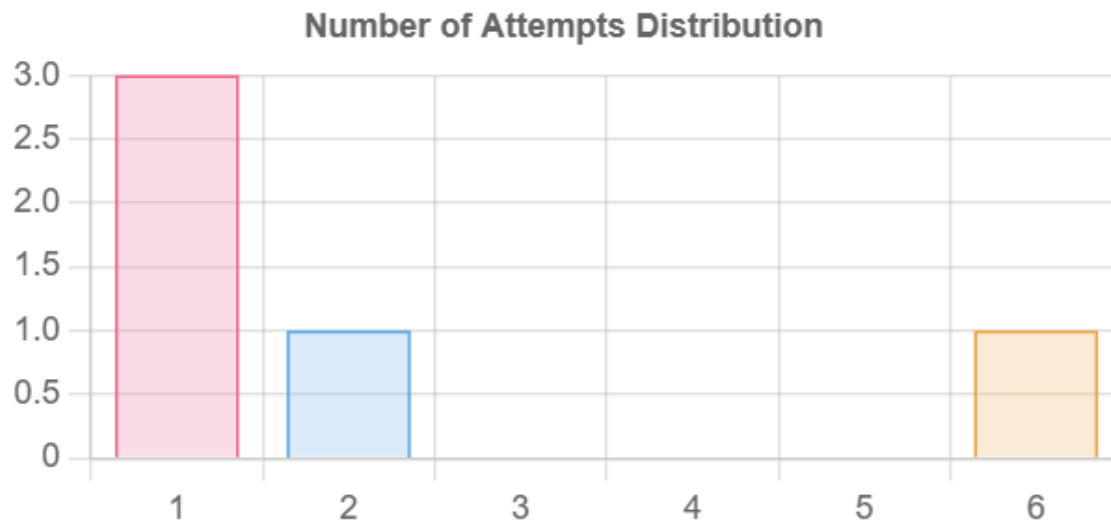


Figure 8 screenshot for feature 3.2 showing the summary statistics and attempt distribution for a certain user

Feature 4.1

This feature allows the user to purchase more game plays when they run out of their three free ones. It interacts with the external REST API, which gives the KRATO\$COIN balance of the logged in user to see if they have enough to buy extra game plays and then also updates the balance of KRATO\$COIN the user has once they purchase some number of game plays. Below are a few screenshots that demonstrate the functionality of this feature. The user that is shown in most of the examples has already used 6 out of their 10 KRATO\$COINs to begin with.

Buy Extra Games

Your KratoSCoin Balance: 4

Your Extra Plays: 0

Number of games to buy:

[Back to Home](#)

Figure 9 screenshot for feature 4.1 showing the page the user is brought to when they click “Buy Extra Games” on the home page, or when they attempt to play a game when they are out of plays.

Buy Extra Games

Your KratoSCoin Balance: 3

Your Extra Plays: 1

Number of games to buy:

You successfully purchased 1 extra game(s).

[Back to Home](#)

Figure 10 screenshot for feature 4.1 showing the buy-games page after the user has successfully purchased an extra game play.

As Figure 10 shows, the user’s balance decrements and their number of extra plays increments after a successful purchase, which is exactly what we would expect.

Buy Extra Games

Your KratoSCoin Balance: None

Your Extra Plays: 0

Number of games to buy:

Buy

[Back to Home](#)

Figure 11 screenshot for feature 4.1 showing the buy-games page for a user that is not registered in the REST API.

Buy Extra Games

Your KratoSCoin Balance: None

Your Extra Plays: 0

Number of games to buy:

Buy

Transaction failed. You may have insufficient funds.

[Back to Home](#)

Figure 12 screenshot for feature 4.1 showing what happens when a user with no KRATO\$COIN tries to make a purchase

Figures 11 and 12 show what happens when logged in as a user that is not registered in the REST API. These users have no KRATO\$COIN and are unable to purchase extra plays.

The video we put together to demonstrate the functionality of our web application shows the sequential flow of a user running out of free plays, purchasing an extra one, then being able to play again until they run out of extra plays.

Project's Learned Lessons

This project heavily relies on object-oriented programming (OOP). Django frameworks build this paradigm into themselves, especially with the models that we create when using Django. For example, our User model extends the AbstractUser class (discussed above), and we defined both attributes and methods that we use to keep track of and utilize User objects. OOP is a big part of this web application development workflow, and we certainly made use of it when creating this project. Another paradigm that we use very frequently in this project is the Model-View-Template structure that Django does so well. We created models for the Users of our application and for each Play of the game, and then we use views to control what gets rendered to the user based on the attributes of the models. And, as is typical with this architecture, we use templates to render the visual components of our webpage. These are the main overarching paradigms that drive the functionality of our web application, but we also used event-driven programming with JavaScript to handle the logic that runs the WorNDly game, RESTful programming to interface with an external API and control the purchasing of new games, and imperative programming throughout the entire application.

One of the most intellectually challenging aspects of this project was making the ability for the website to limit a user's daily plays and purchase additional plays. It had to handle a lot of interaction between the REST api in order to purchase the new plays, the Python code to check if the user is able to play, and then the javascript code to actually tell the user if they are able to play or not and disable the game when necessary. Another challenge was the profile page and getting the information from the SQLite database through python and then giving it through the views.py file to the html file, which showed the information about each play. Also, the data about counts for each number of attempts needed to get grabbed by the JavaScript to then create the bar graph.

I think that our tendency to work together on the project in person was the most important factor to our success. We did not previously have a ton of experience working with version control and the Git workflow, so figuring that piece out together and making sure we were on the same page while developing was very important to our success. We also did a good job of delegating tasks between partners, and the split between which features each group member was working on was clear and even.

☐	USER
☐	d
☐	c
☐	superuser
☐	b
☐	ezra
☐	yo
☐	test2
☐	hello
☐	sgisiner
☐	test
☐	dgallag5
11 users	

Figure 13 screenshot showing that there are at least 10 users

☐	PLAY
☐	Play object (12)
☐	Play object (11)
☐	Play object (10)
☐	Play object (9)
☐	Play object (8)
☐	Play object (7)
☐	Play object (6)
☐	Play object (5)
☐	Play object (4)
☐	Play object (3)
☐	Play object (2)
☐	Play object (1)
12 plays	

Figure 14 screenshot showing that there are at least 10 plays